

DISEÑO ARQUITECTÓNICO

Traductor Braille

Arquitectura implementada (AS-IS)

Sistema	Aplicación web estática de traducción español ↔ braille
Estilo	Monolito modular del lado cliente organizado por capas
Versión	1.1 · Revisión enfocada en arquitectura · 20 de julio de 2026

1. Alcance de la arquitectura

Este documento describe exclusivamente la arquitectura implementada en el proyecto entregado. Se analizan la estructura del sistema, las capas, los módulos, sus dependencias, los contratos internos, los flujos de ejecución y el despliegue.

Presentamos una **Arquitectura modular por capas dentro de un monolito cliente**. Todos los módulos se entregan juntos y se ejecutan en el navegador.

1.1 Límites del sistema

- Dentro del sistema: HTML, CSS, módulos ES, DOM, reglas de traducción y tablas braille.
- Fuera del sistema: el usuario y el servicio de impresión del navegador/sistema operativo.

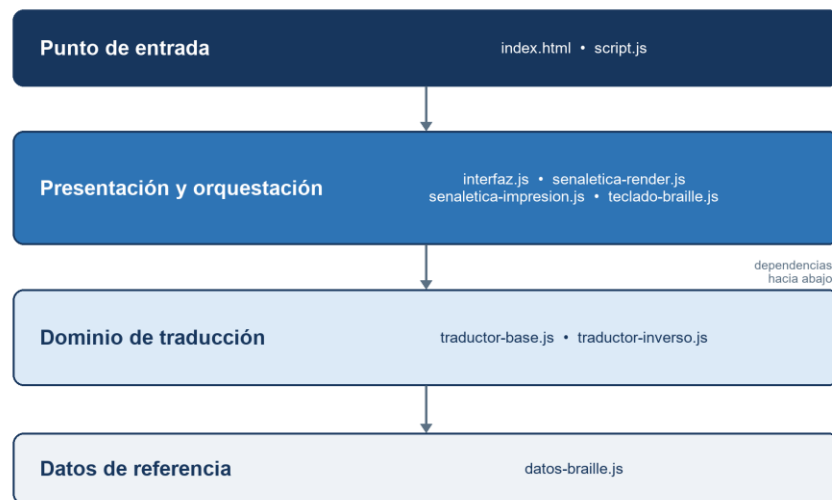
Figura 1. Contexto arquitectónico



2. Vista lógica por capas

Las capas son lógicas, no servidores separados. La regla principal es que las dependencias avanzan desde la entrada y presentación hacia el dominio y, finalmente, hacia los datos de referencia.

Figura 2. Organización lógica de la solución



2.1 Responsabilidad de cada capa

Capa	Responsabilidad arquitectónica	Módulos
Entrada	Construir la página, cargar estilos y arrancar la aplicación.	index.html; style.css; script.js
Presentación / aplicación	Gestionar DOM, eventos, estado temporal, visualización, teclado e impresión.	interfaz.js; senaletica-render.js; senaletica-impresion.js; teclado-braille.js
Dominio	Aplicar las reglas de traducción directa e inversa sin construir HTML.	traductor-base.js; traductor-inverso.js

Capa	Responsabilidad arquitectónica	Módulos
Datos de referencia	Definir patrones de seis puntos, signos, prefijos y mapas inversos.	datos-braille.js

3. Vista de componentes

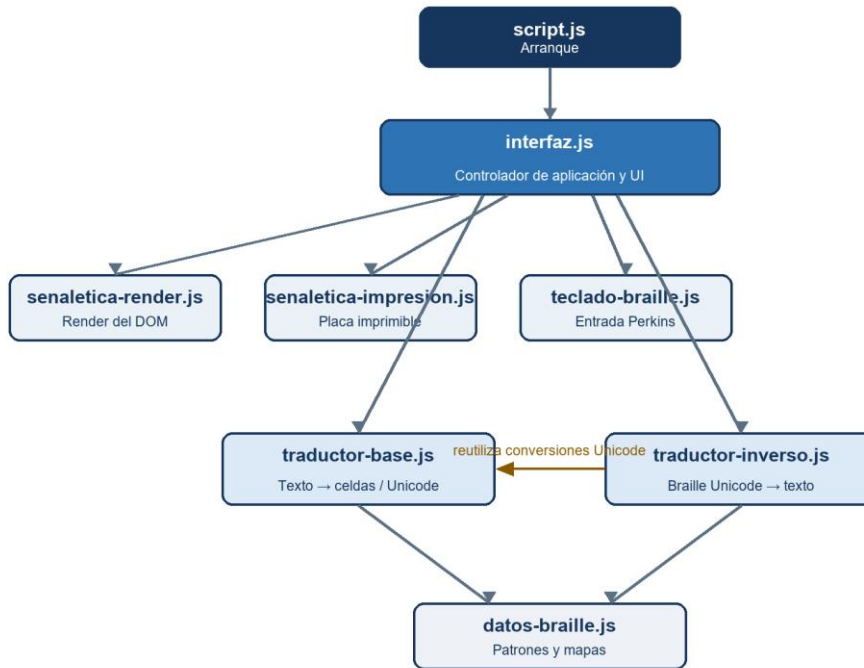
Componente	Rol	Servicios expuestos / efecto
script.js	Bootstrap	Importa iniciarApp() y ejecuta el arranque.
interfaz.js	Controlador	Coordina eventos, estado y casos de uso de ambas traducciones.
traductor-base.js	Servicio de dominio	textoABraille(), conversiones Unicode y salidas para pantalla/impresión.
traductor-inverso.js	Servicio de dominio	brailleATexto() y composición inversa.
datos-braille.js	Repositorio en memoria	Mapas de letras, acentos, signos, dígitos y prefijos.
senaletica-render.js	Adaptador visual	Crea cuadratines y líneas dentro del DOM.
senaletica-impresion.js	Adaptador de salida	Prepara la placa, aplica espejo y llama window.print().
teclado-braille.js	Adaptador de entrada	Traduce FDS/JKL y teclas de control a callbacks.

3.1 Fronteras de responsabilidad

- interfaz.js puede conocer el DOM y los servicios de dominio; el dominio no puede conocer la interfaz.
- senaletica-render.js solo transforma el modelo lógico en elementos visuales.
- senaletica-impresion.js encapsula la dependencia con la API de impresión del navegador.
- teclado-braille.js recibe callbacks; no ejecuta directamente reglas de traducción.

4. Vista de dependencias

Figura 3. Grafo de dependencias entre módulos



4.1 Dependencias observadas

Origen	Depende de	Motivo
script.js	interfaz.js	Inicialización de la aplicación.
interfaz.js	traductor-base.js	Traducción directa, Unicode y creación de celdas.
interfaz.js	traductor-inverso.js	Traducción braille → texto.
interfaz.js	render, impresión y teclado	Adaptadores de entrada y salida.
traductor-base.js	datos-braille.js	Patrones y reglas de correspondencia.
traductor-inverso.js	datos-braille.js y traductor-base.js	Mapa inverso y conversiones Unicode.
senaletica-impresion.js	traductor-base.js	Cadena Unicode específica para impresión.

5. Contratos internos y modelo de intercambio

5.1 Servicios de dominio

Contrato	Entrada	Salida	Consumidor
textoABraille(texto)	string	{ celdas, desconocidos }	interfaz.js; pruebas
celdasAUnicodePantalla(celdas)	object[]	string Unicode	interfaz.js; pruebas
celdasAUnicodeImpresion(celdas)	object[]	string Unicode	senaletica-impresion.js
brailleATexto(entrada)	string Unicode	{ texto, desconocidos }	interfaz.js; pruebas
dotsAUnicode(dots)	boolean[6]	carácter U+2800–U+283F	interfaz.js; traductor-inverso.js

5.2 Modelo lógico de celda

Tipo	Estructura	Significado arquitectónico
cell	{ type, dots[6], label, special?, mayusculaDoble? }	Unidad traducida independiente de su representación visual.
space	{ type: 'space' }	Separación de palabras.
newline	{ type: 'newline' }	Separación de líneas.
unknown	{ type: 'unknown', char }	Entrada sin mapeo; permite advertir sin romper el flujo.

Este modelo intermedio es la principal frontera entre el dominio y la presentación: el traductor produce datos; los adaptadores deciden cómo mostrarlos o imprimirlos.

6. Vista dinámica

Figura 4. Flujos principales de ejecución

Flujo A · Texto → Braille



Flujo B · Braille → Texto



Ambos flujos consultan datos-braille.js; el estado existe solo durante la sesión de página.

6.1 Traducción directa

1. interfaz.js obtiene el texto y valida que no esté vacío.
2. textoABraille() recorre caracteres y aplica estados de número y mayúscula.
3. datos-braille.js proporciona los patrones de seis puntos.
4. El resultado se representa como celdas lógicas y recuento de desconocidos.
5. La interfaz delega el renderizado, Unicode y la preparación de impresión.

6.2 Traducción inversa

La entrada puede provenir de Unicode pegado o del teclado Perkins simulado. brailleATexto() interpreta prefijos numéricos y de mayúscula, consulta el mapa inverso y devuelve texto más el recuento de celdas desconocidas.

7. Vista de despliegue

Figura 5. Despliegue físico



Ejecución local: sin API remota, autenticación, sesión de servidor ni persistencia.

Los archivos pueden publicarse en cualquier servidor estático. El navegador descarga una única aplicación y ejecuta toda la lógica localmente. Node.js y Jest son herramientas de desarrollo, no componentes del despliegue productivo.

8. Decisiones arquitectónicas

Decisión	Razón	Consecuencia
JavaScript ES Modules sin framework	El alcance es reducido y no requiere una plataforma adicional.	Despliegue liviano; la coordinación manual crece con nuevas funciones.
Modelo lógico antes del DOM	Separar reglas braille de la forma de visualizarlas.	Dominio reutilizable y comprobable mediante pruebas unitarias.
Mapas braille centralizados	Evitar duplicar patrones en traductores y UI.	Una fuente única de verdad, pero versionada dentro del código.
Estado solo en memoria	No existe requisito de historial o cuenta de usuario.	Privacidad y simplicidad; el estado se pierde al recargar.
Impresión mediante navegador	Aprovechar infraestructura disponible en el cliente.	No requiere backend; el resultado depende del motor de impresión.

8.1 Conformidad de la implementación

- La dirección principal de dependencias respeta las capas definidas.
- Los módulos de dominio pueden ejecutarse en pruebas sin DOM.
- La suite Jest aprobó 21 de 21 pruebas de dominio y recorridos de ida y vuelta.

- La mayor concentración arquitectónica está en interfaz.js, que reúne ambos traductores y la coordinación de impresión.

9. Conclusión arquitectónica

La implementación corresponde a una arquitectura modular por capas del lado cliente. La separación más importante se encuentra entre la coordinación visual, el dominio de traducción y los datos de referencia braille.

El sistema no usa React, Next.js, TypeScript, Tailwind, MVC, caché ni backend. Introducir esos elementos en el documento describiría una arquitectura hipotética y no el proyecto entregado.

Para su tamaño actual, la arquitectura es coherente. La evolución natural consiste en dividir interfaz.js por casos de uso y extraer las conversiones Unicode compartidas, manteniendo intactas las fronteras entre presentación, dominio y datos de referencia.